With that hook in place, RSpec will *conditionally* load spec/support/db.rb if (and only if) any examples are loaded that have a :db tag.

Now, you'll need to go add the tag to your specs. Both your acceptance and integration specs need it at the end of the RSpec.describe line, just before the do keyword. First, spec/acceptance/expense_tracker_api_spec.rb:

06-integration-specs/08/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb
```
RSpec.describe 'Expense Tracker API', :db do
```

Next, spec/integration/app/ledger_spec.rb:

06-integration-specs/08/expense_tracker/spec/integration/app/ledger_spec.rb
```
RSpec.describe Ledger, :aggregate_failures, :db do
```

While you're at it, you can also remove the require_relative '../../support/db' line from ledger_spec.rb.

At last, you can run RSpec with the same random seed and get just the single failure we were expecting:

```
$ bundle exec rspec --seed 32043
« truncated »

Finished in 0.05786 seconds (files took 0.22818 seconds to load)
11 examples, 1 failure, 1 pending

Failed examples:

rspec ./spec/integration/app/ledger_spec.rb:33 # ↩
ExpenseTracker::Ledger#record when the expense lacks a payee rejects the ↩
expense as invalid

Randomized with seed 32043
```

We're back down to just one spec failing predictably. It's time to continue the Red/Green/Refactor cycle by adding just enough behavior to your object to make the spec pass.

Filling In the Behavior

The failing spec is expecting the Ledger class's record method to return error information if we pass in an invalid expense (one with no payee defined). Add the following highlighted lines to the record definition in app/ledger.rb:

06-integration-specs/08/expense_tracker/app/ledger.rb
```
module ExpenseTracker
  RecordResult = Struct.new(:success?, :expense_id, :error_message)

  class Ledger
    def record(expense)
      unless expense.key?('payee')
        message = 'Invalid expense: `payee` is required'
```